
INF1350

Sistemas Reativos

NodeMCU - ESP8266

Prof.: Adriano Branco

abranco@inf.puc-rio.br

Slides baseados em aulas anteriores da Prof. Noemi Rodriguez

microcontrolador ESP8266-12F

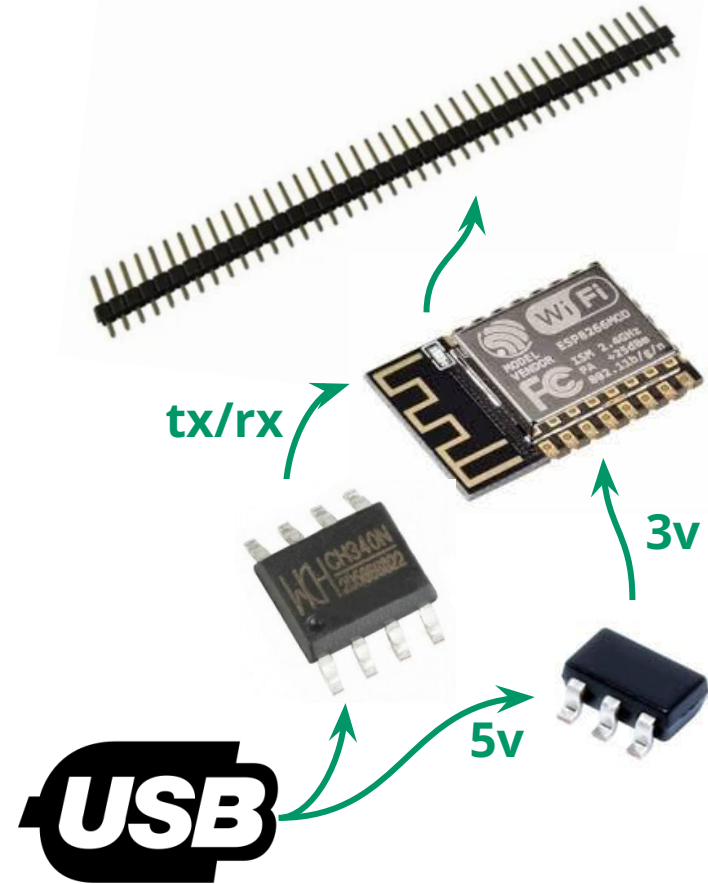
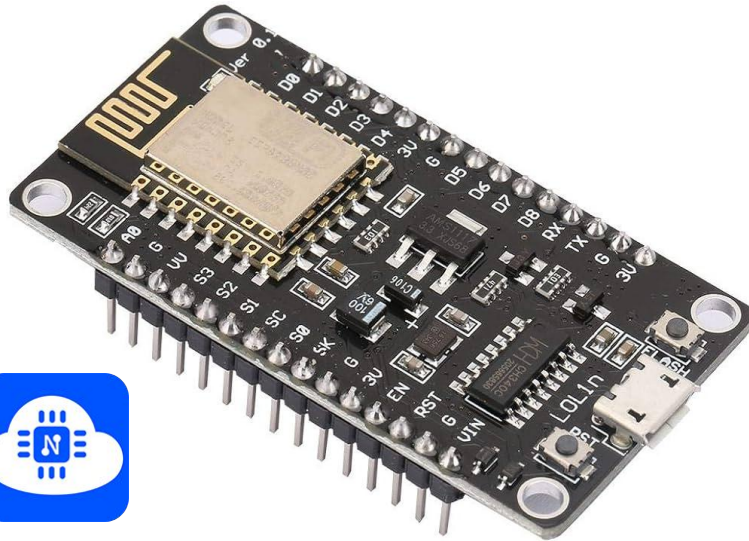
- **CPU de 32 bits, 80 MHz**
- 64 KB ROM (boot)
- 32 KB RAM p/ instruções
- 80 KB RAM p/ dados
- 4MB Flash
- 2.4 GHz wifi - 802.11 b/g/n
- GPIOs, ADC, PWM
- I²C, SPI, I²S, UART



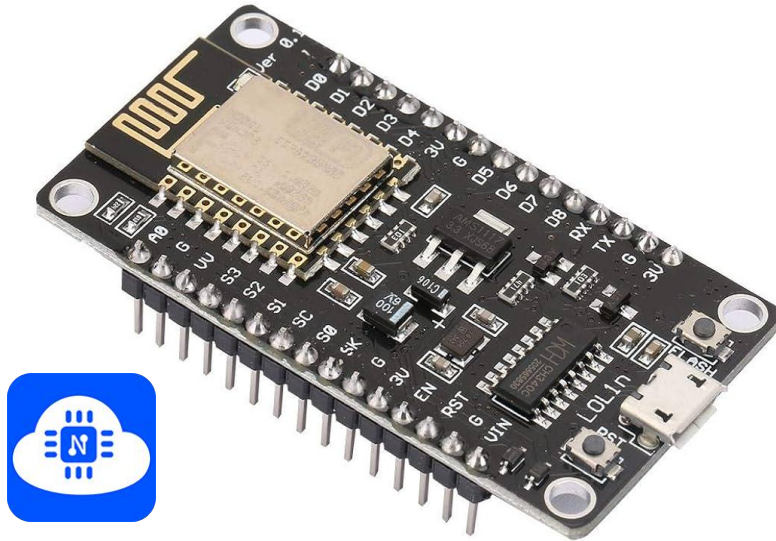
Hardware: NodeMCU



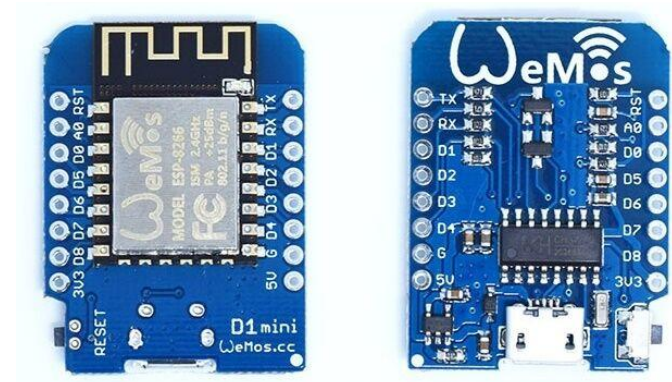
NodeMCU



Hardware: NodeMCU & D1 Mini



NodeMCU



D1 Mini
(WeMos/LOLIN)

ESP8266-12F vs Arduino Uno (ATmega328P)

ESP8266-12F	ATmega328P
CPU de 32 bits, 80 MHz	CPU de 8 bits, 16MHz
64 KB ROM (boot) 32 KB RAM p/ instruções	32 KB Flash p/ instruções
80 KB RAM p/ dados	2K SRAM p/ dados
4MB Flash armazenamento/instruções	1 KB EEPROM armazenamento
2.4 GHz wifi - 802.11 b/g/n	----
GPIOs, ADC, PWM	GPIOs, ADC, PWM
I ² C, SPI, I ² S, UART	I ² C, SPI, UART

Programação ESP8266-12F



RTOS SDK (☐ ESP-IDF)
NonOS SDK (congelado)
ambos em manutenção



NodeMCU



sobre o NonOS SDK



Arduino IDE / PlatformIO
sobre o NonOS SDK



MicroPython

Espruino (JavaScript, comunidade)

Forth, Lisp, BASIC: projetos descontinuados

sobre o NonOS SDK

Programação ESP8266-12F



RTOS SDK (☐ ESP-IDF)
NonOS SDK (congelado)
ambos em manutenção



NodeMCU



sobre o NonOS SDK



Arduino IDE / PlatformIO
sobre o NonOS SDK



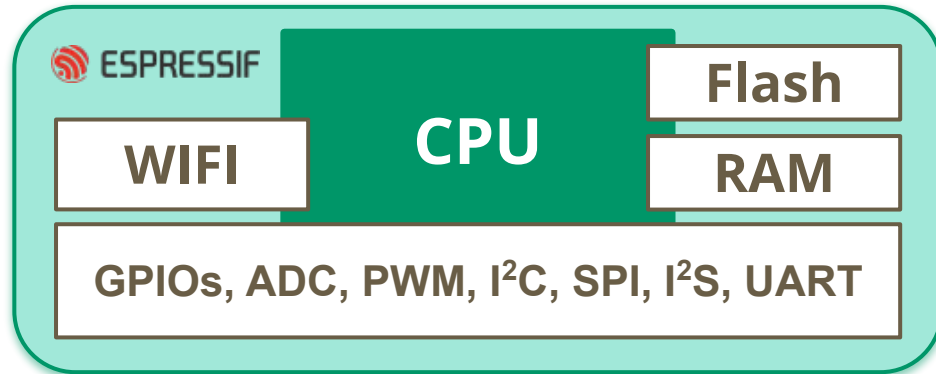
MicroPython

Espruino (JavaScript, comunidade)

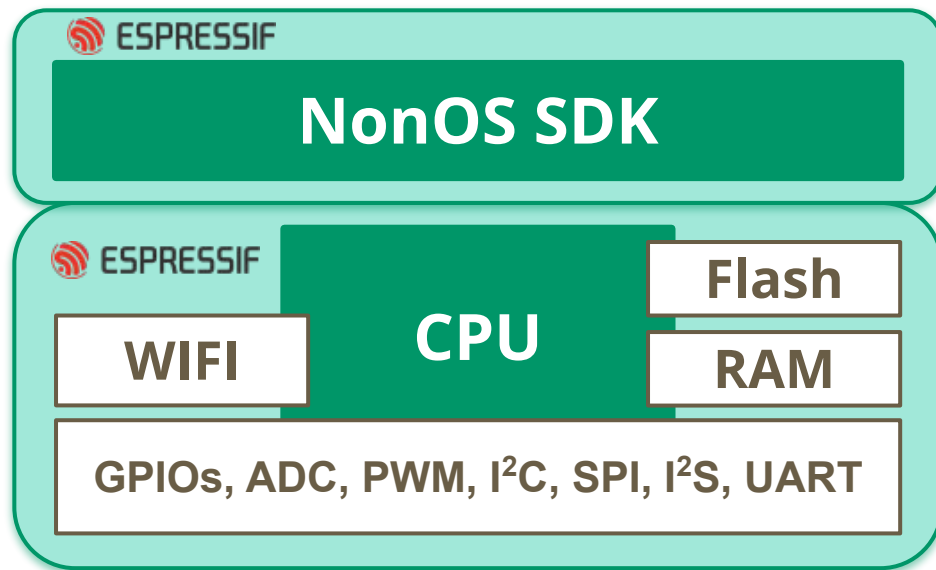
Forth, Lisp, BASIC: projetos descontinuados

sobre o NonOS SDK

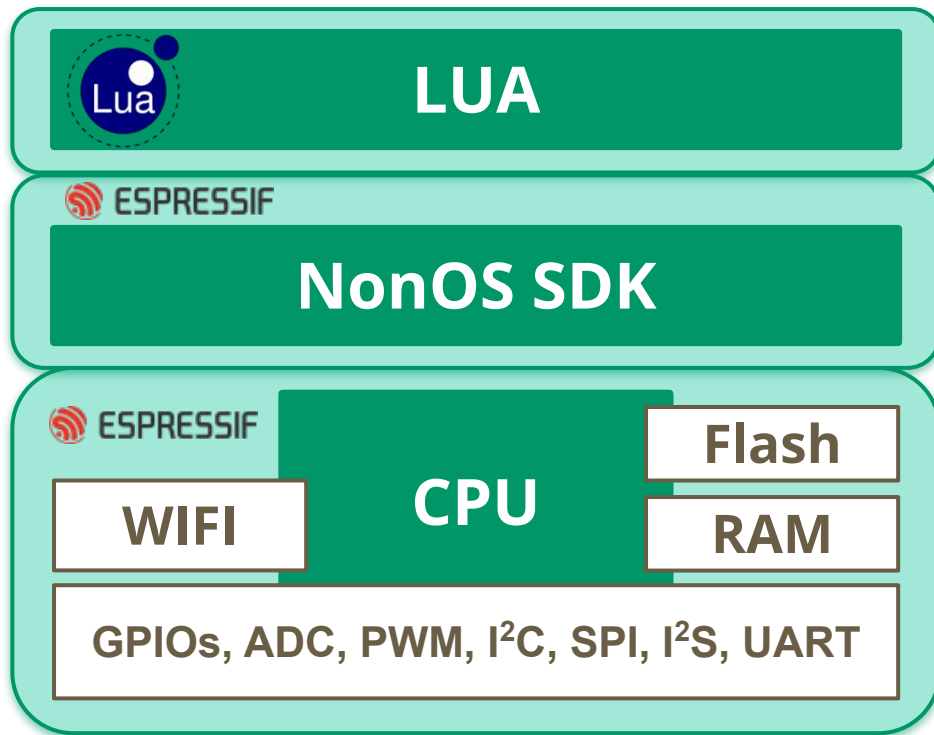
ESP8266-12F + SDK Espressif + NodeMCU/Lua



ESP8266-12F + SDK Espressif + NodeMCU/Lua

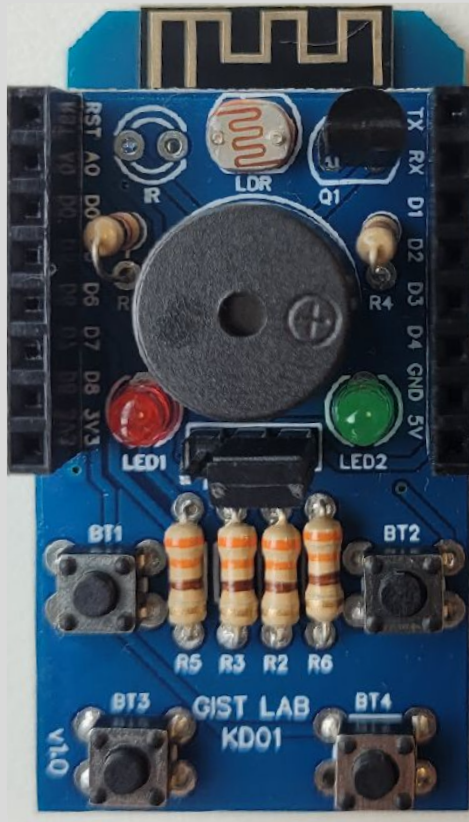


ESP8266-12F + SDK Espressif + NodeMCU/Lua

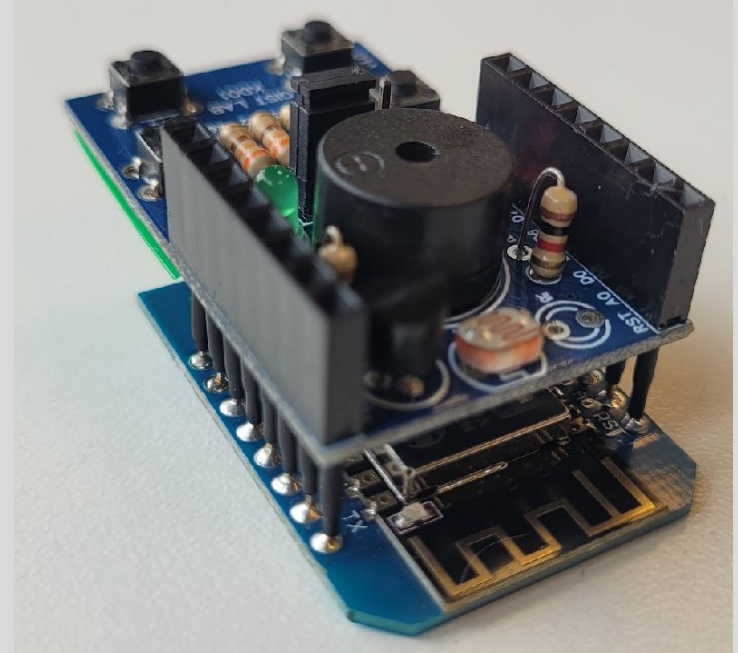
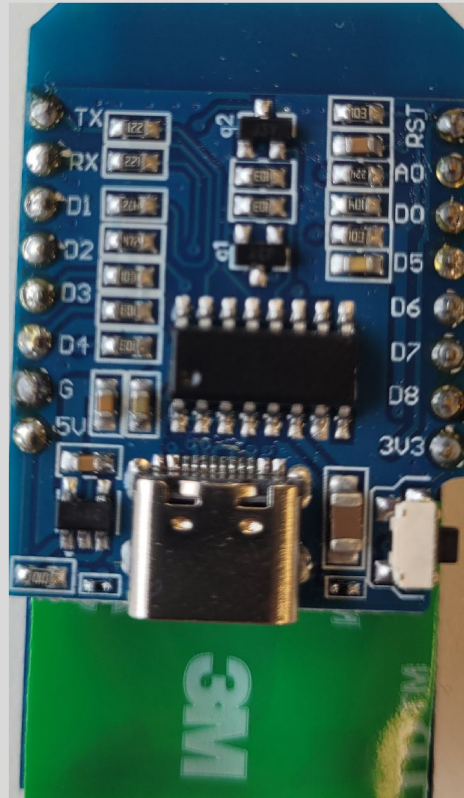


Placa da Disciplina: D1 Mini + Kit Didático

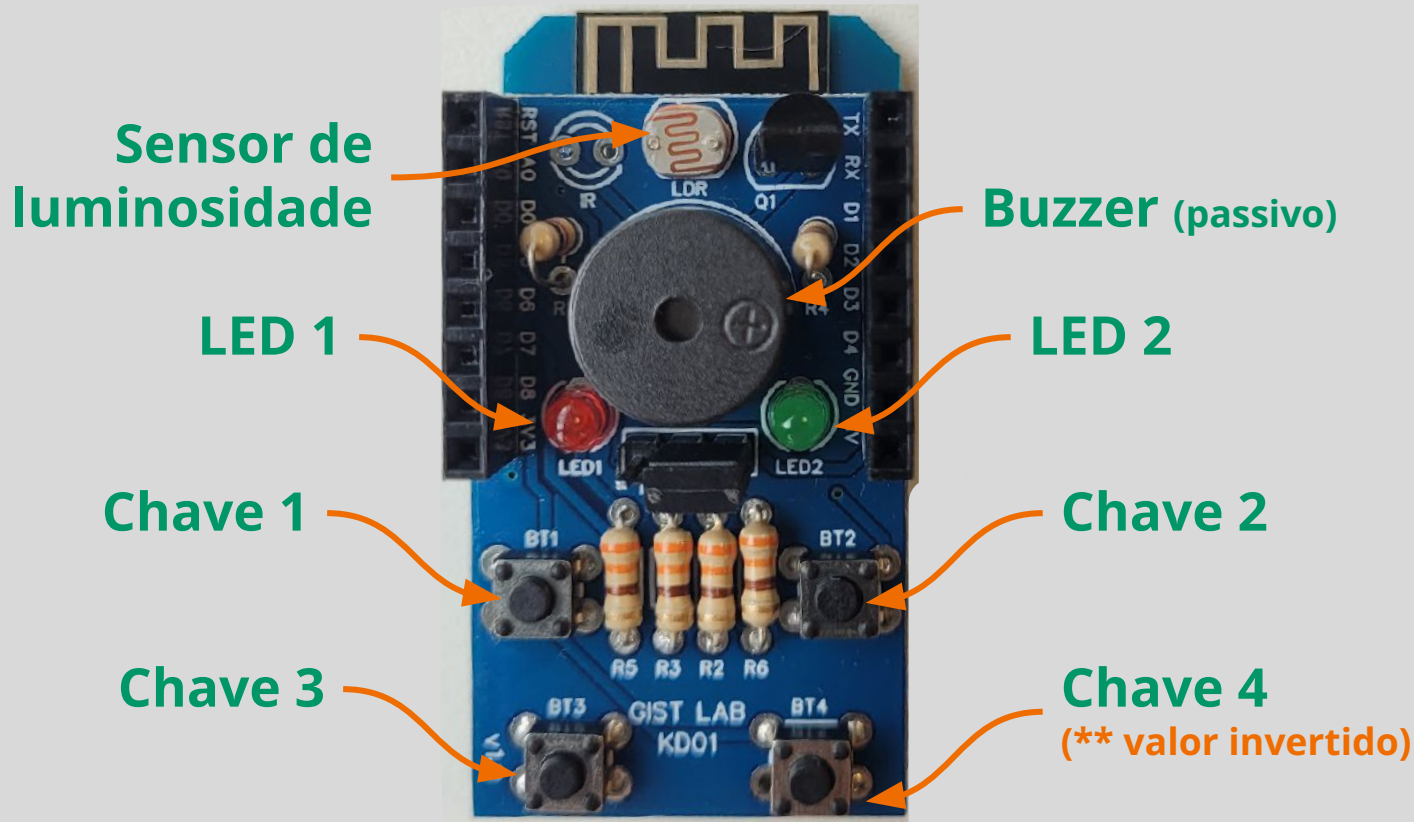
Vista superior



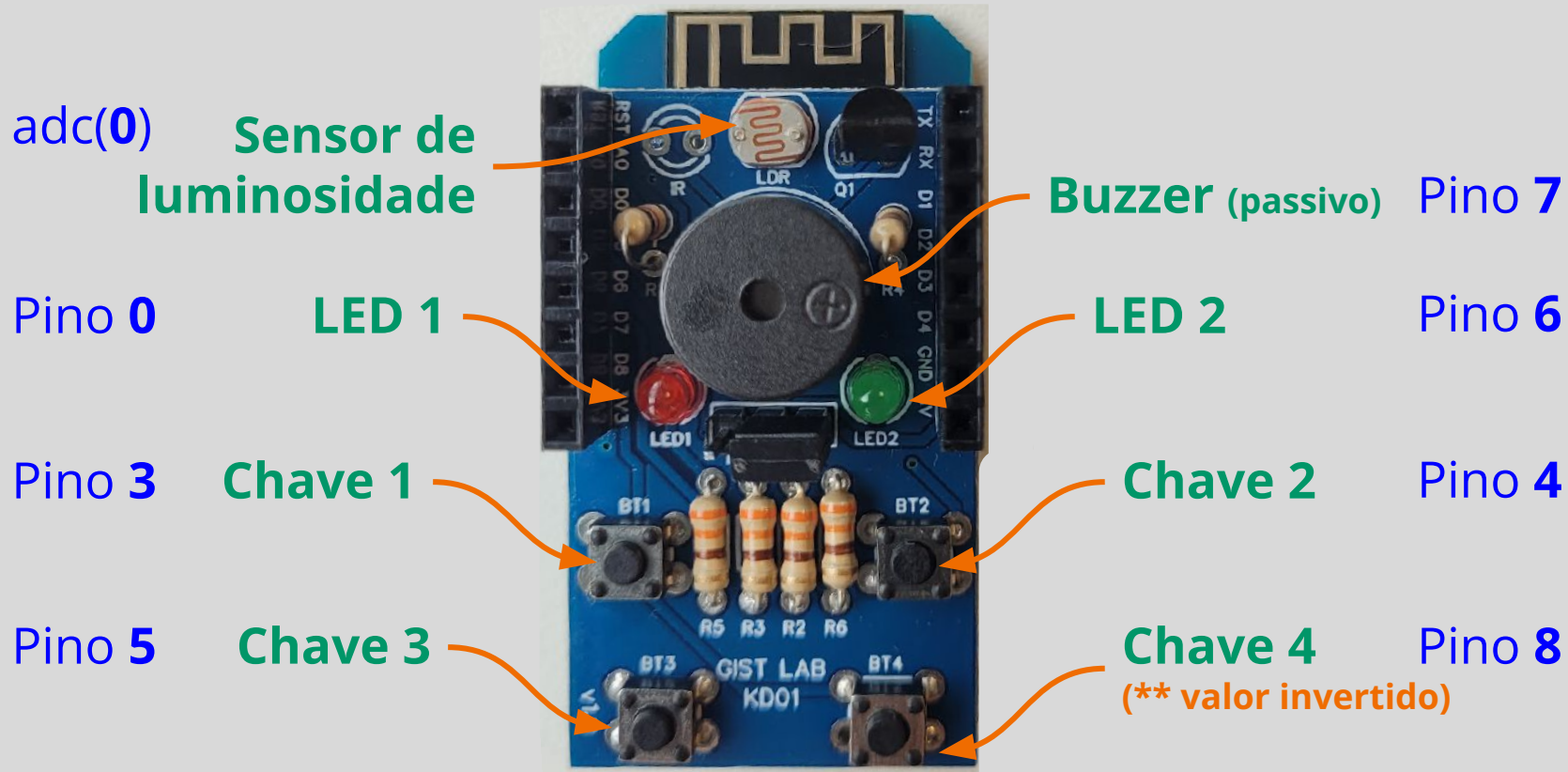
Vista inferior



Componentes KD01

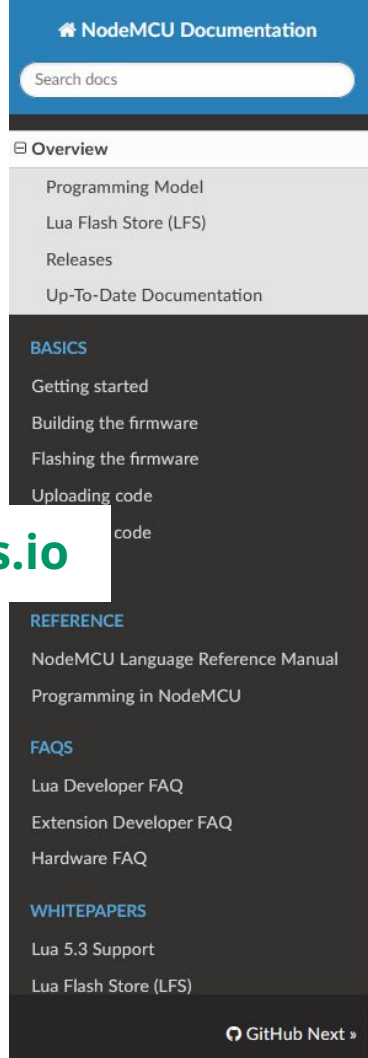


Componentes KD01



Documentação NodeMCU Lua

nodemcu.readthedocs.io



[Docs](#) » Overview

[Edit on GitHub](#)

NodeMCU Documentation

NodeMCU is an open source [Lua](#) based firmware for the [ESP8266 WiFi SOC from Espressif](#) and uses an on-module flash-based [SPIFFS](#) file system. NodeMCU is implemented in C and is layered on the [Espressif NON-OS SDK](#).

The firmware was initially developed as is a companion project to the popular ESP8266-based [NodeMCU development modules](#), but the project is now community-supported, and the firmware can now be run on *any* ESP module.

→ [Getting Started](#)

Programming Model

The NodeMCU programming model is similar to that of [Node.js](#), only in Lua. It is asynchronous and event-driven. Many functions, therefore, have parameters for callback functions. To give you an idea what a NodeMCU program looks like study the short snippets below. For more extensive examples have a look at the [/lua_examples](#) folder in the repository on GitHub.

```
-- a simple HTTP server
srv = net.createServer(net.TCP)
srv:listen(80, function(conn)
  conn:on("receive", function(sck, payload)
    print(payload)
    sck:send("HTTP/1.0 200 OK\r\nContent-Type: text/html\r\n\r\n<h1> Hello, NodeMCU. </h1>")
  end)
  conn:on("sent", function(sck) sck:close() end)
end)
```

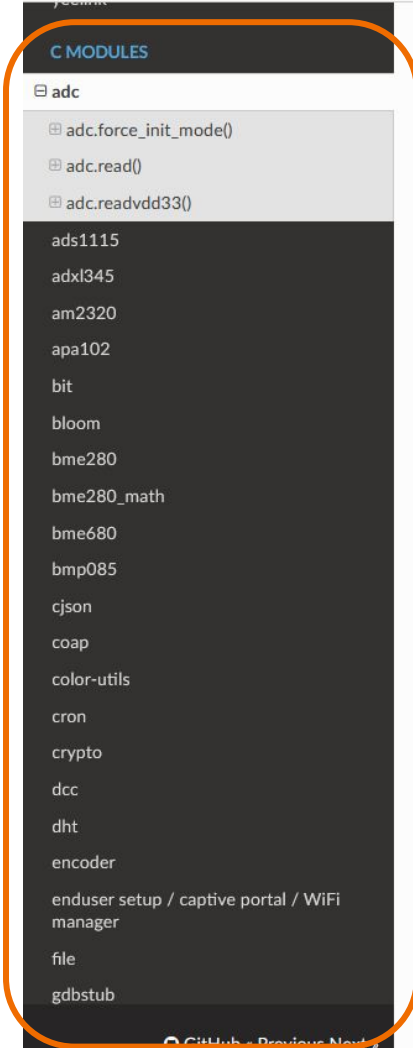
```
-- connect to WiFi access point (DO NOT save config to flash)
wifi.setmode(wifi.STATION)
station_cfg={}
station_cfg.ssid = "SSID"
station_cfg.pwd = "password"
station_cfg.save = false
wifi.sta.config(station_cfg)
```

Documentação NodeMCU

Módulos C

OBS:

Os módulos devem ser previamente selecionados na hora de gerar o firmware.



Docs » C Modules » adc

[Edit on GitHub](#)

ADC Module

Since	Origin / Contributor	Maintainer	Source
2014-12-24	Zeroday	jmatsson	adc.c

The ADC module provides access to the in-built ADC.

On the ESP8266 there is only a single-channel, which is multiplexed with the battery voltage. Depending on the setting in the "esp init data" (byte 107) one can either use the ADC to read an external voltage, or to read the system voltage (vdd33), but not both.

Which mode to use the ADC in can be configured via the `adc.force_init_mode()` function. Note that after switching from one to the other a system restart (e.g. power cycle, reset button, `node.restart()`) is required before the change takes effect.

<code>adc.force_init_mode()</code>	Checks and if necessary reconfigures the ADC mode setting in the ESP init data block.
<code>adc.read()</code>	Samples the ADC.
<code>adc.readvdd33()</code>	Reads the system voltage.

adc.force_init_mode()

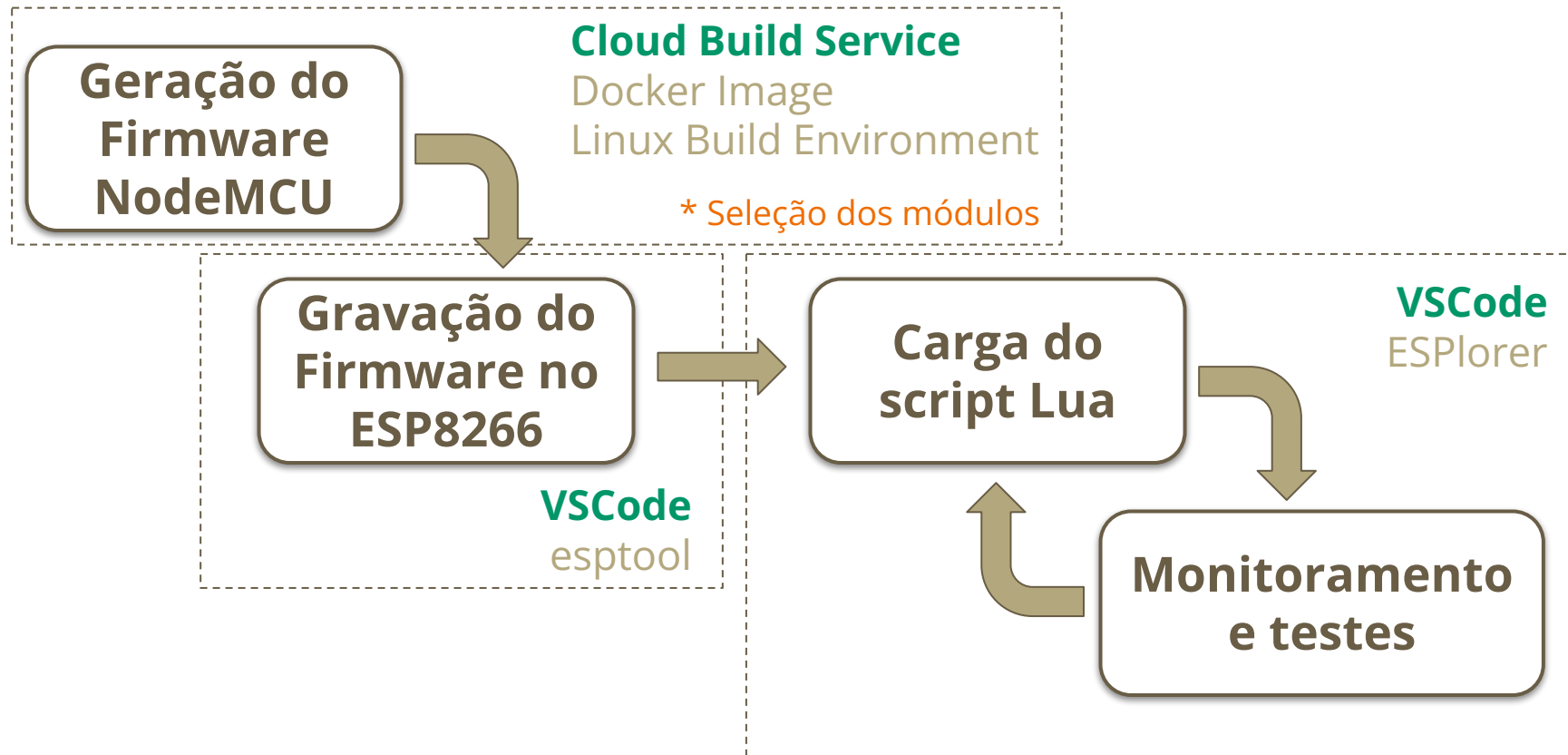
Checks and if necessary reconfigures the ADC mode setting in the ESP init data block.

Syntax

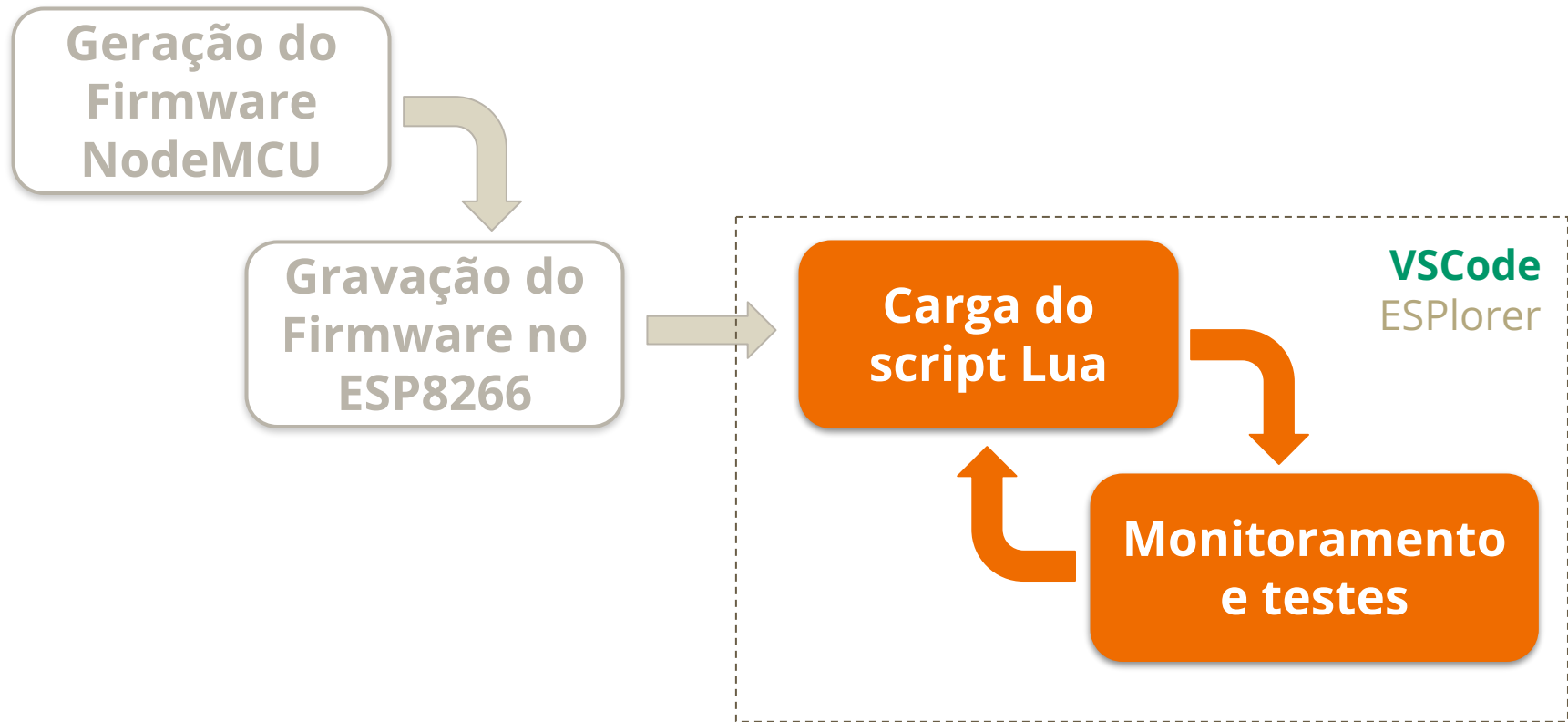
```
adc.force_init_mode(mode_value)
```

Parameters

Programação do Dispositivo



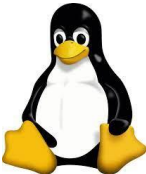
Programação do Dispositivo



Módulos incluídos no firmware atual

- **adc**
- bit
- file
- **gpio**
- http
- **mqtt**
- net
- node
- **pwm**
- pwm2
- sjson
- sntp
- struct
- **timer**
- uart
- **wifi**

Uso do VSCode (NodeMCU-Tools) ou ESPlorer

	Sala de Aula LabGrad	Seu computador
	  ESPlorer VSCode	  ESPlorer VSCode
		  ESPlorer VSCode

VSCode: o que instalar

1. Extensão **NodeMCU-Tools** (boressoft)
 - terminal, upload/download de arquivos e gravação do firmware
 - Obs: Abrir uma pasta na IDE antes de tentar conectar.
2. Extensão **Lua** (sumneko) — destaque de sintaxe e autocompletar
 - Ctrl+Shift+P → **Lua: Open Addon Manager**
 - ativar o addon **nodemcu-esp8266** (funções e documentação do NodeMCU)
3. Windows: driver USB-serial da placa (**CH340**)
 - geralmente o Windows instala sozinho; se a porta COM não aparecer, instalar o **CH341SER.EXE** (site da WCH: wch-ic.com)
 - conferir no Gerenciador de Dispositivos: **USB-SERIAL CH340 (COMx)**
4. Linux: permissão de acesso à USB (ver próximo slide)

ESPlorer (<https://esp8266.ru/esplorer/>)

1. Baixar o arquivo ZIP
<https://github.com/4refr0nt/ESPlorer/releases/download/v0.2.0/ESPlorer-0.2.0.zip>
2. Extrair arquivos numa pasta do usuário
3. Executar na própria pasta o script .bat, tem que ter Java 8+ instalado.
4. Se desejar, configure um “atalho” no seu desktop.

Para autorizar acesso à usb no Linux:

- `sudo usermod -a -G dialout $USER`
- Para funcionar, precisa refazer login ou reiniciar.

Para remover conflito com BRLTTY em algumas distros Linux:

- `sudo apt-get remove brlTTY`

Interface do ESPLORER

ESPLORER v0.2.0-rc1 by 4refr0nt

File Edit ESP View Links ?

NodeMCU+Mi

Scripts Cor

Open R

init.lua time

```
1 print('
2 function
3 print
4 conn
5 conn
6 pri
7 pri
8 cor
9 end)
10 cor
11 cor
12
13
14
15 end
16 tmr.alarm(0, 1000, 1, function()
17   if wifi.sta.getip()==nil then
18     print("connecting to AP...")
19   else
20     print('ip: ',wifi.sta.getip())
21     getHTTPreq()
22     tmr.stop(0)
23   end
24 end)
25
```

Always On Top

✓ Show Log Ctrl+L

Clear log Ctrl+F1

Clear terminal Ctrl+F2

✓ Show toolbar at top left

Show Extra buttons at left ← COMING SOON

✓ Show Snippets panel at right

✓ Show FileManager panel at right

✓ Show Extra buttons at bottom right

I'm already make donation, please hide dontation button at bottom right!

Terminal font size inc Ctrl+NumPad +

Terminal font size dec Ctrl+NumPad -

Editor font size inc Alt+NumPad +

Editor font size dec Alt+NumPad -

Log font size inc Ctrl+Alt+NumPad +

Log font size dec Ctrl+Alt+NumPad -

Reset all font size to default

COM3

Open CTS

DTR RTS

Close

AutoScroll

BOL

115200

```
00001060 C2 1C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001070 22 01 53 68 65 6C 6C 45 78 65 63 75 74 65 57 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001080 53 48 45 4C 4C 33 32 2E 64 6C 6C 00 6A 02 47 65 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001090 74 53 74 61 72 74 75 70 49 6E 66 6F 57 00 DB 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000010A0 48 65 61 70 53 65 74 49 6E 66 6F 72 6D 61 74 69 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000010B0 6F 6E 00 00 48 45 52 4E 45 4C 33 32 2E 64 6C 6C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000010C0 00 00 8F 00 5F 5F 77 67 65 74 6D 61 69 6E 61 72 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000010D0 67 73 00 00 53 00 5F 5F 43 5F 73 70 65 63 69 6E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000010E0 69 63 5F 68 61 6E 64 6C 65 72 00 00 52 00 5F 58 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000010F0 63 70 74 46 69 6C 74 65 72 00 FF 00 5F 65 78 69 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001100 74 00 B3 00 5F 63 65 78 69 74 00 00 20 04 65 78 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001110 69 74 00 00 71 03 5F 77 63 6D 64 6C 6E 00 6C 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001120 5F 69 6E 69 74 74 65 72 6D 00 A0 00 5F 61 6D 73 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001130 67 5F 65 78 69 74 00 00 82 00 5F 5F 73 65 74 75 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001140 73 65 72 6D 61 74 68 65 72 72 00 00 C4 00 5F 63 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001150 6F 6D 6F 64 65 00 00 18 01 5F 66 6D 6F 64 65 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001160 00 00 80 00 5F 5F 73 65 74 5F 61 70 70 5F 74 79 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00001170 70 65 00 00 30 00 3F 74 65 72 6D 69 6E 61 74 65 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Format

FS Info

Reload

decode64.lua

init.lua

test10k.txt

test20K.bin

time.lua

write.exe

recv.an't open file") and end _dump()

recv._dump=nil<CR><LF>

Operation done. Duration = 1140 ms

5files collectgarbage Date Snippet3 Snippet4 Snippet5 Snippet6 Snippet7 Snippet8 Snippet9 Snippet10 Snippet11 Snippet12 Snippet13 Snippet14 Snippet15

Heap Chip Info Chip ID Flash ID Reset

Save to ESP Send to ESP Run Upload

node.heap()

Send

CR

LF

Donate

Dicas sobre a IDE

- **Recomendado:** VSCode com a extensão “NodeMCU-Tools”, no Windows da sala ou no seu computador (Windows ou Linux).
- **Alternativa:** ESPlorer, para upload, execução e monitoramento.
- O editor do ESPlorer é bem “fraco”: se usá-lo, edite o código em outra IDE.
- No Linux da sala (LabGrad) não há acesso à USB.

Arquivo “init.lua”

- Após um reset, o sistema vai procurar um arquivo com o nome `init.lua` para executar na inicialização. Normalmente colocamos nesse arquivo a configuração do WIFI e a chamada para execução do nosso programa.
-  Caso aconteça algum erro na inicialização do nosso programa, o sistema vai ficar em loop reiniciando automaticamente. Impedindo qualquer ação para corrigir o problema.
- Uma opção é não criar o arquivo `init.lua`.
- Outra opção é incluir no início do `init.lua` um timer de alguns segundos antes de executar nosso programa, com isso teremos tempo para apagar ou renomear o arquivo.

Programando o NodeMCU

Programando o NodeMCU

- GPIO
 - Configurar modo do pino: leitura, escrita ou trigger/interrupção
 - Ler ou escrever no pino ou executar função de trigger
- ADC
 - Ler o valor do ADC
- TIMER
 - Configurar tempo e modo de funcionamento
 - Ativar, desativar ou executar função de disparo do timer
- PWM
 - Configurar frequência e largura do pulso
 - Ativar e desativar
- WIFI
 - Configurar conexão como **Station** ou **AP**
- MQTT
 - Configurar conexão ao servidor
 - Assinar canais de dados
 - Publicar nos canais de dados

Programando o NodeMCU - entrada e saída

```
local led1 = 0
```

```
local led2 = 6
```

```
local sw1 = 3
```

```
local sw2 = 4
```

```
local sw3 = 5
```

```
local sw4 = 8
```

```
gpio.mode(led1, gpio.OUTPUT)
```

```
gpio.mode(led2, gpio.OUTPUT)
```

```
-- apaga os leds
```

```
gpio.write(led1, gpio.LOW);
```

```
gpio.write(led2, gpio.LOW);
```

Mode:

gpio.OUTPUT

gpio.INPUT

gpio.INT (interrupt mode)

Troca bit

`gpio.write(led1, bit.band(gpio.read(led1)+1,1))` Lê o valor do pino

`gpio.write(led1, bit.band(gpio.read(led1)+1,1))` Soma com 1

`gpio.write(led1, bit.band(gpio.read(led1)+1,1))` Faz “bit and” 0x01

`gpio.write(led1, bit.band(gpio.read(led1)+1,1))` Escreve de volta

Alternativa:

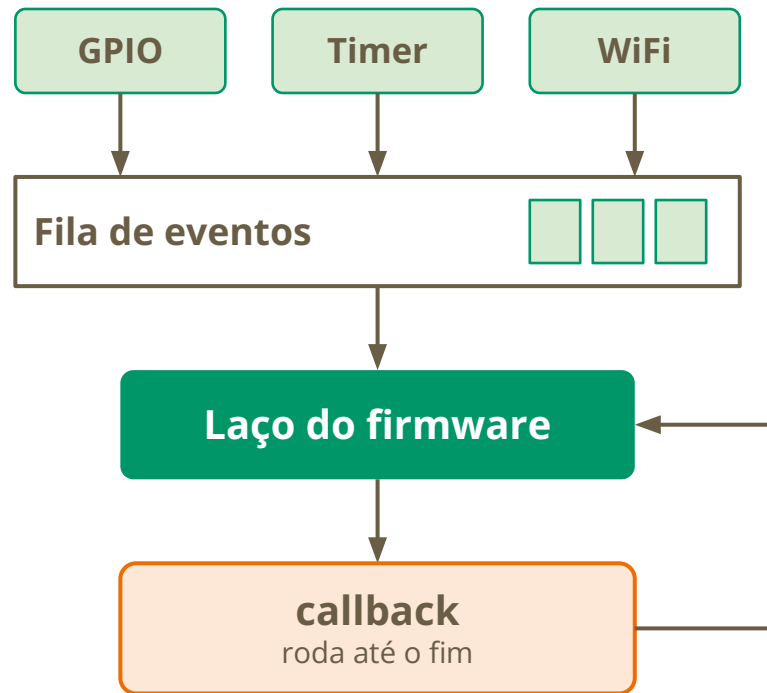
`gpio.write(led1, ((gpio.read(led1)==1) and 0) or 1)`

Programando o NodeMCU - Reativo (callback)

- Existem diversos tipos de eventos pré-definidos como:
 - Pino GPIO mudou de estado.
 - Conexão ou desconexão do WIFI.
 - Timer
 - Vários eventos específicos para cada módulo.
- É o programador que define que função que vai ser executada para cada tipo de evento.

Modelo de execução do NodeMCU

- Um único laço de eventos, controlado pelo firmware (você não o escreve).
- Cada evento (pino, timer, WiFi...) coloca seu callback numa fila.
- Os callbacks rodam um de cada vez, até o fim, sem interrupção.
- Enquanto um callback executa, nada mais é tratado, nem o WiFi.



Consequência: callbacks devem ser curtos

✗ Estilo Arduino: espera ocupada

```
local led1 = 0
gpio.mode(led1, gpio.OUTPUT)
while true do
    gpio.write(led1,
    bit.band(gpio.read(led1)+1,1))
    tmr.delay(500000) -- espera 0,5 s
end
```

O laço nunca devolve o controle ao firmware: nenhum outro evento é tratado e, após alguns segundos, o watchdog reseta a placa.

✓ Estilo reativo: timer

```
local led1 = 0
gpio.mode(led1, gpio.OUTPUT)
local function pisca()
    gpio.write(led1,
    bit.band(gpio.read(led1)+1,1))
end
local t = tmr.create()
t:alarm(500, tmr.ALARM_AUTO, pisca)
```

O callback termina rápido e o firmware volta ao laço; o timer agenda a próxima execução.

Regra: nunca espere dentro de um callback;
divida o trabalho em eventos (timers, triggers).

Exemplo - callback para interrupção de pino (trigger)

```
local led1 = 0
local sw1 = 3
gpio.mode(led1, gpio.OUTPUT)
gpio.mode(sw1,gpio.INT,gpio.PULLUP)

function trocaled(level,timestamp)
    gpio.write(led1, bit.band(gpio.read(led1)+1,1))
end

-- cadastra a função trocaled para ser chamada se
-- a chave 1 for apertada
gpio.trig(sw1, "down", trocaled)
```

Parâmetros de uma callback de trigger.



Exemplo - callback para interrupção de pino

```
local led1 = 0
```

```
local sw1 = 3
```

```
gpio.mode(led1, gpio.OUTPUT)
```

```
gpio.mode(sw1,gpio.INT,gpio.PULLUP)
```

```
gpio.trig(sw1, "down",
```

```
    function (level,timestamp)
```

```
        gpio.write(led1, bit.band(gpio.read(led1)+1,1))
```

```
    end)
```

Codificação
alternativa

Debouncing - comparação de tempo

```
local led1 = 0
```

```
local sw1 = 3
```

```
local delay = 200000 --em microsegundos!
```

```
local last = 0
```

```
gpio.mode(led1, gpio.OUTPUT)
```

```
gpio.mode(sw1,gpio.INT,gpio.PULLUP)
```

```
function trocaled(level,timestamp)
```

```
    if timestamp - last < delay then return end
```

```
    last = timestamp
```

```
    gpio.write(led1, bit.band(gpio.read(led1)+1,1))
```

```
end
```

```
gpio.trig(sw1, "down", trocaled)
```

NodeMCU - Interface timer

É um exemplo de “objeto” em Lua.

Interface

```
t = tmr.create()  
t:register() → t:alarm()  
t:start() →  
t:stop()  
t:unregister()  
... 
```

tobj:alarm()

Syntax

```
tobj:alarm(interval_ms, mode, func())
```

Parameters

- `interval_ms` timer interval in milliseconds. Maximum value is 6870947 (1:54:30.947).
- `mode` timer mode:
 - `tmr.ALARM_SINGLE` a one-shot alarm (and no need to call [unregister\(\)](#))
 - `tmr.ALARM_SEMI` manually repeating alarm (call [start\(\)](#) to restart)
 - `tmr.ALARM_AUTO` automatically repeating alarm
- `func(timer)` callback function which is invoked with the timer object as an argument

Returns

`true` if the timer was started, `false` on error

NodeMCU - pisca-pisca

```
local led1 = 0
local led2 = 6
local meusleds = {led1, led2}

for _,ledi in ipairs (meusleds) do
    gpio.mode(ledi, gpio.OUTPUT)
    gpio.write(ledi, gpio.LOW);
end

local function piscapisca (t)
    for _,led in ipairs (meusleds) do
        gpio.write(led, bit.band(gpio.read(led)+1,1))
    end
end

-- cria e dispara timer
local mytimer = tmr.create()
mytimer:register(1000, tmr.ALARM_AUTO, piscapisca)
mytimer:start()
```

Parâmetros de uma
callback de timer.



NodeMCU - pisca-pisca versão 2

```
local led1 = 0
local led2 = 6

local function disparapiscapisca (led, tempo)
    local function piscapisca(timer)
        gpio.write(led, bit.band(gpio.read(led)+1,1))
    end
    -- coloca o pino dos leds em modo de saída
    gpio.mode(led, gpio.OUTPUT)
    -- apaga o led
    gpio.write(led, gpio.LOW);
    tmr.create():alarm(tempo, tmr.ALARM_AUTO, piscapisca)
end

-- cada led pisca em tempo diferente
disparapiscapisca (led1, 500)
disparapiscapisca (led2, 750)
```

Debouncing: outra maneira

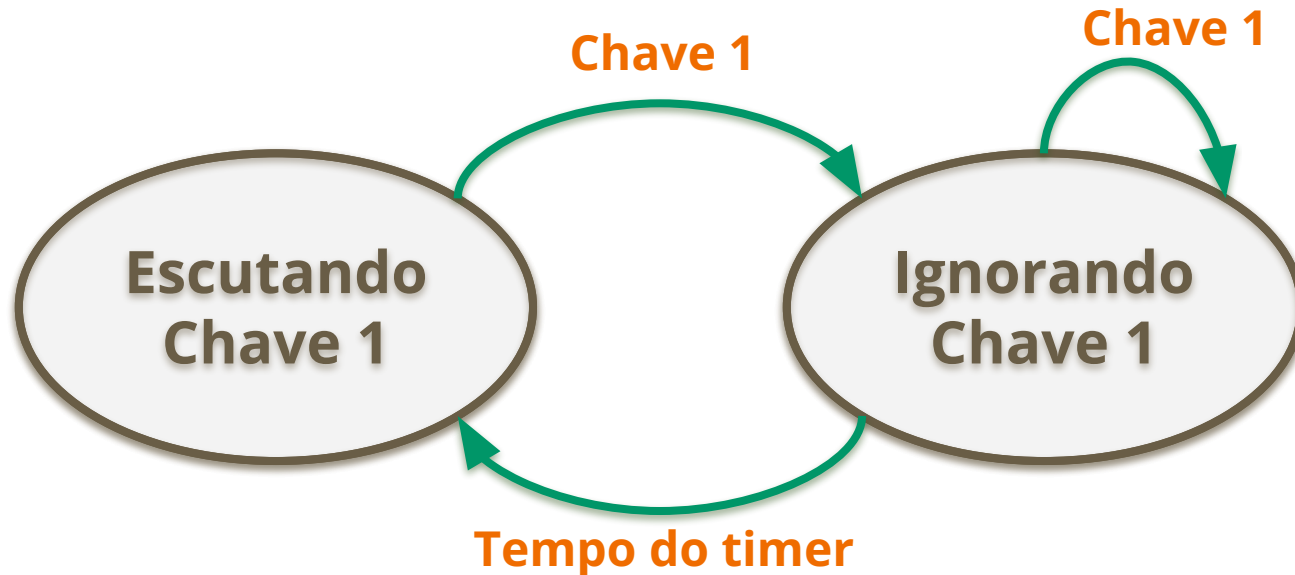
Alterar o estado do programa

... para não reagir ao botão por algum tempo.

Desabilitar o trigger!!!

`gpio.trig(sw1, "down", cbchave)`  `gpio.trig(sw1)`

Máquina de estados



Debouncing - com timer

```
local led1 = 0
local sw1 = 3
gpio.mode(led1, gpio.OUTPUT)
gpio.mode(sw1,gpio.INT,gpio.PULLUP)

function trocaled(level,timestamp)
    gpio.trig(sw1)
    gpio.write(led1, bit.band(gpio.read(led1)+1,1))
    tmr.create():alarm(200, tmr.ALARM_SINGLE,
        function(t)
            gpio.trig(sw1, "down", trocaled)
        end)
end

gpio.trig(sw1, "down", trocaled)
```

Exercício

1. Programa pisca-pisca que para de piscar quando qualquer chave é pressionada.
2. Programa que pisca os 2 leds. A chave 1 faz o led1 parar de piscar e recomeçar, e a chave 2 faz o led2 parar de piscar e recomeçar.
3. (velha conhecida) Programa que pisca led vermelho. O pressionamento da chave 1 faz o led piscar mais rápido e o da chave 2 faz o led piscar mais devagar. O pressionamento conjunto das chaves 1 e 2 dentro do intervalo de 500ms faz o programa parar para sempre (com leds apagados).

Códigos dos slides:

<https://www.inf.puc-rio.br/~abranco/sr-23.2/code/nodemcu/>